

Protección y Seguridad en Servicios de IAM

Contexto y relevancia

La **gestión de identidad y acceso (IAM)** es un pilar crítico en la seguridad de entornos cloud. Sin embargo, su implementación no solo implica definir permisos o roles, sino también **proteger los servicios que gestionan estas identidades**. En esta sesión, nos enfocamos en cómo asegurar servicios de directorio en la nube, específicamente en AWS, y su integración con sistemas empresariales como **Active Directory (AD)**.

Objetivo de la sesión

El objetivo principal es enseñar a:

- Configurar y proteger **servicios de directorio en AWS (como AWS Directory Service)**.
- Integrar estos servicios con sistemas on-premises (ej: Active Directory).
- Aplicar mecanismos de seguridad avanzados, como cifrado, autenticación multifactor (MFA) y monitoreo continuo.

Conceptos clave

1. Servicios de directorio en la nube :

- Herramientas como **AWS Directory Service** permiten gestionar usuarios, grupos y políticas de acceso en la nube, similar a un Active Directory tradicional.
- Ejemplo: Un directorio en AWS puede autenticar usuarios para acceder a recursos como EC2, RDS o aplicaciones empresariales.

2. Integración con Active Directory :

- Muchas organizaciones usan AD local. AWS permite extender su funcionalidad a la nube mediante servicios como **AD Connector** o **AWS Managed Microsoft AD**.

3. Seguridad en autenticación y autorización :

- Mecanismos como **MFA**, cifrado de datos (AWS KMS) y políticas de acceso mínimo garantizan que solo usuarios autorizados acceden a recursos críticos.

Estructura de la sesión

La sesión se divide en:

- **Configuración de AWS Directory Service** : Creación de directorios y buenas prácticas.
- **Integración con Active Directory** : Sincronización segura entre entornos on-premises y cloud.
- **Mecanismos de seguridad** : Cifrado, MFA, monitoreo con CloudTrail y CloudWatch.

Ejemplos prácticos

Supongamos una empresa que migra sus aplicaciones a AWS y necesita autenticar empleados mediante su AD existente:

1. **Paso 1** : Crear un directorio en AWS Directory Service.
2. **Paso 2** : Configurar una **relación de confianza (trust)** entre AD on-premises y el directorio en AWS.
3. **Paso 3** : Habilitar MFA para administradores y monitorear accesos con CloudTrail.

Resultado : Los usuarios acceden a recursos en la nube con las mismas credenciales de AD, pero con capas adicionales de seguridad.

Importancia teórica y práctica.

- **Teórica** : Entender cómo los servicios de directorio en la nube cumplen con normativas como **GDPR** o **ISO 27001** .
- **Práctica** : Aprender a usar herramientas como **AWS Directory Service** , **AD Connector** y **CloudWatch** para mitigar riesgos.

Esta sesión sienta las bases para proteger la infraestructura de identidad en la nube, combinando teoría con ejemplos técnicos. Los conceptos aquí explicados son esenciales para diseñar arquitecturas seguras y escalables.

Implementación de Servicios de Directorio en AWS

¿Qué es AWS Directory Service?

AWS Directory Service es un conjunto de herramientas gestionadas que permiten crear y administrar directorios en la nube, integrándose con recursos de AWS (como EC2, RDS, o aplicaciones) y sistemas empresariales. Su objetivo es centralizar la autenticación y autorización de usuarios, grupos y servicios.

Componentes clave :

1. Microsoft AD administrado por AWS :

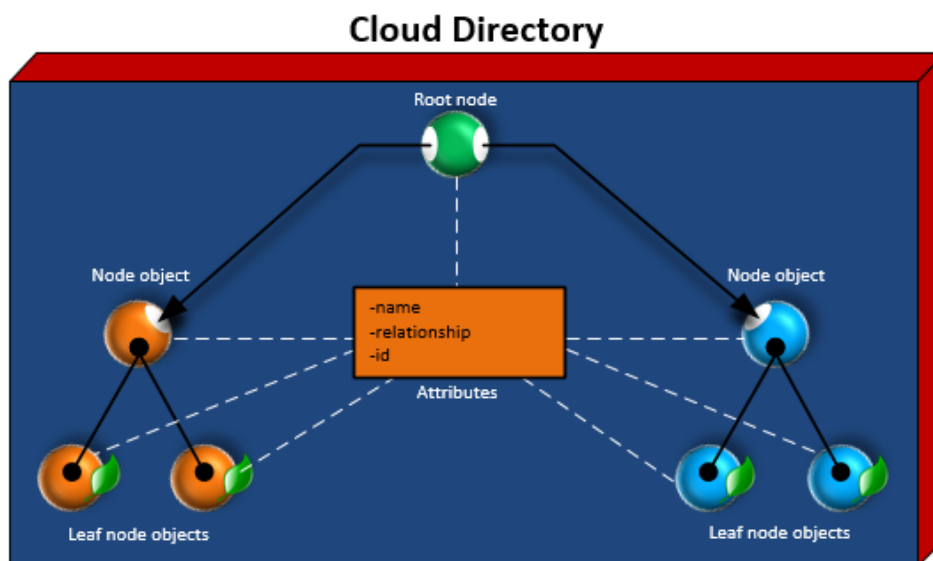
- Directorio gestionado por AWS, compatible con **Active Directory (AD)** de Microsoft.
- Ideal para entornos empresariales que requieren autenticación mediante protocolos como **LDAP** , **Kerberos** o **NTLM** .
- Escalable y altamente disponible, con réplicas en múltiples zonas de disponibilidad.

2. Anuncio simple :

- Versión ligera y económica de un directorio compatible con LDAP.
- Diseñado para pequeñas organizaciones o casos de uso básicos (ej: autenticación de usuarios en aplicaciones web).
- No se admiten características avanzadas de AD, como relaciones de confianza (*trusts*) o políticas de grupo.

3. Conector AD :

- Proxy seguro para conectar un **Active Directory local** con AWS.
- Permite a los usuarios locales acceder a recursos en AWS sin replicar datos.
- Útil para migraciones híbridas o empresas con infraestructura existente.



Creación de un Directorio en AWS

Paso a paso :

1. **Elegir el tipo de directorio :**
 - **AWS Managed Microsoft AD** : Para entornos empresariales complejos.
 - **AD simple** : Para necesidades básicas.
 - **Conector AD** : Para integración híbrida.
2. **Definir parámetros :**
 - **Nombre de dominio** : Ejemplo: . **corp.example.com**
 - **Tamaño** : Pequeño (100-500 usuarios) o Grande (500+ usuarios).
 - **Rojo (VPC)** : Seleccione la VPC y subredes donde se desplegará el directorio.
3. **Configuración de seguridad :**
 - **Cifrado** : Usar **AWS KMS** para cifrar datos en reposo.
 - **Grupos de seguridad** : Restringir el acceso al directorio solo a IPs o servicios autorizados.

Ejemplo de CLI :

```
1 # Crear un directorio AWS Managed Microsoft AD
2 aws ds create-microsoft-ad \
3   --name corp.example.com \
4   --password "SecurePassword123!" \
5   --edition Enterprise \
6   --vpc-settings VpcId=vpc-12345678,SubnetIds=subnet-abc123,subnet-def456
```

Caso de uso: Microsoft AD administrado por AWS

Escenario : Una empresa necesita autenticar usuarios de AWS y recursos locales mediante un directorio único.

- **Solución :**
 1. Cree un **AWS Managed Microsoft AD** en la VPC de AWS.
 2. Configurar una **relación de confianza** con el AD local.
 3. Usar **grupos de seguridad** para permitir acceso solo desde redes autorizadas.

Beneficios :

- Los usuarios acceden a recursos en la nube y locales con las mismas credenciales.
- Gestión centralizada de políticas y permisos.

Seguridad en la Implementación

- **Cifrado :**
 - Datos en reposo: Protegidos mediante **AWS KMS** .
 - Datos en tránsito: Comunicación cifrada con **TLS 1.2** .
- **Control de acceso :**
 - Usar **roles de IAM** para restringir quién puede administrar el directorio.
 - Aplicar el **principio de privilegio mínimo** .

Errores Comunes y Buenas Prácticas

- **Errores :**
 - No restringir Grupos de Seguridad, permitiendo acceso público al directorio.
 - Usar contraseñas débiles durante la creación del directorio.
- **Buenas prácticas :**
 - Habilitar **MFA** para administradores del directorio.
 - Monitorear actividades con **AWS CloudTrail** .

Configuración de Relaciones de Confianza (Fideicomisos)

Una **relación de confianza** permite que dos directorios (ej: AWS Managed AD y AD on-premises) autenticuen usuarios entre sí. Es esencial para migraciones híbridas o acceso unificado a recursos.

Tipos de Relaciones de Confianza

1. **Unidireccional :**

- Un directorio confía en el otro, pero no al revés.
- Ejemplo: Los usuarios locales acceden a recursos en AWS, pero no viceversa.

2. **Bidireccional :**

- Ambos directorios se autentican mutuamente.
- Ejemplo: Los usuarios de AWS acceden a recursos locales y viceversa.

Pasos para Configurar una Relación de Confianza

Requisitos previos :

- Conectividad segura entre entornos (VPN o AWS Direct Connect).
- DNS configurado para resolver nombres entre ambos directorios.
- Permisos administrativos en ambos AD.

Proceso :

1. **Crear un AWS Managed Microsoft AD :**

- Si no existe, cree un directorio en AWS (ej:).**aws.corp.example.com**

2. **Configurar DNS :**

- Asegurar que el AD local pueda resolver el dominio de AWS y viceversa.
- Usar **Route 53** o servidores DNS locales para registros condicionales.

3. **Crear la relación de confianza en AWS :**

- Navegar a **AWS Directory Service > Directorios** > Seleccionar el directorio.
- En la pestaña **Relaciones de confianza** , hacer clic en **Crear relación de confianza** .
- Especificar:
 - **Dominio remoto** : Nombre del AD local (ej :).**onprem.corp.example.com**
 - **Tipo de confianza** : Unidireccional o bidireccional.
 - **Contraseña de confianza** : Debe coincidir en ambos lados.

4. Configurar la relación de confianza local :

- En el servidor AD local, utilice **dominios y confianzas de Active Directory** .
- Agregue un nuevo **dominio de confianza** y siga los pasos para sincronizar con el directorio de AWS.

Ejemplo práctico con CLI

Crear una relación de confianza desde AWS :

```
1 aws ds create-trust \  
2   --directory-id d-1234567890 \  
3   --remote-domain-name onprem.corp.example.com \  
4   --trust-type Forest \  
5   --trust-direction Two-Way \  
6   --trust-password "SecureTrustPass123!"
```

Verificar la relación de confianza :

```
1 aws ds describe-trusts --directory-id d-1234567890
```

Seguridad en Relaciones de Confianza

- **Cifrado** : Las comunicaciones entre directorios usan **Kerberos AES-256** .
- **Firewalls** : Asegurar que los puertos requeridos (ej: 53/DNS, 88/Kerberos) estén abiertos.
- **Monitoreo** : Usar **AWS CloudTrail** para auditar cambios en las relaciones de confianza.

Errores comunes

- **DNS no resuelve** : Si los dominios no se encuentran, la confianza fallará.
- **Contraseñas no coinciden** : La contraseña de confianza debe ser idéntica en ambos lados.
- **Permisos insuficientes** : El usuario debe tener rol de **Administrador de Empresa** en AD.

Beneficios de las Relaciones de Confianza

- **Acceso unificado** : Los usuarios locales acceden a recursos AWS (ej: EC2, RDS) sin credenciales separadas.
- **Simplifica migraciones** : permite la transición gradual a la nube.
- **Cumplimiento normativo** : Centraliza el control de acceso para auditorías.

Las relaciones de confianza son esenciales para integrar entornos híbridos, pero requieren una configuración precisa de DNS, seguridad y permisos. AWS Directory Service simplifica este proceso, aunque es clave validar cada paso para evitar errores críticos.

La implementación de servicios de directorio en AWS requiere seleccionar el tipo adecuado (Managed AD, Simple AD o AD Connector) y aplicar medidas de seguridad desde el diseño. Estos servicios son la base para integrar identidades empresariales con la nube, garantizando autenticación segura y escalable.

Configuración de AWS KMS para Cifrado en AWS Directory Service

AWS Key Management Service (KMS) permite crear y gestionar claves de cifrado para proteger datos en reposo. En el caso de **AWS Directory Service** , el cifrado se aplica a:

- Datos del directorio (usuarios, grupos, políticas).
- Contraseñas almacenadas.
- Registros de auditoría.

Pasos para configurar KMS

Requisitos previos :

1. Crear una clave KMS :

- Ir a **AWS KMS > Claves administradas por el cliente > Crear clave** .
- Seleccione tipo de clave: **Simétrica** (recomendado para datos en reposo).
- Definir alias (ej:).**directory-encryption-key**

2. Permisos IAM :

- Asignar políticas para que **AWS Directory Service** pueda usar la clave.
- Ejemplo de política:

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Effect": "Allow",  
6       "Principal": {"Service": "ds.amazonaws.com"},  
7       "Action": "kms:GenerateDataKey",  
8       "Resource": "*"   
9     }  
10  ]  
11 }
```

Crear un Directorio con Cifrado KMS

Usando AWS CLI :

```
1 # Crear un directorio AWS Managed Microsoft AD con cifrado KMS  
2 aws ds create-microsoft-ad \  
3   --name corp.example.com \  
4   --password "SecurePassword123!" \  
5   --edition Enterprise \  
6   --vpc-settings VpcId=vpc-12345678,SubnetIds=subnet-abc123,subnet-def456 \  
7   --kms-key-id arn:aws:kms:us-east-1:123456789012:key/abcd1234-a123-4567-8abc-def123456789
```

Parámetros clave :

- **--kms-key-id**: ARN de la clave KMS creada previamente.

Verificación del Cifrado

1. En la consola de AWS :

- Navegar a **AWS Directory Service** > Seleccionar el directorio > Ver detalles.
- Confirmar que el campo **Encryption** muestra el ARN de la clave KMS.

2. Con AWS CLI :

```
1 aws ds describe-directories --directory-ids d-1234567890
```

Seguridad y Buenas Prácticas

1. Rotación automática de claves :

- Habilitar rotación anual en KMS para cumplir con los estándares de seguridad.

2. Control de acceso :

- Usar **políticas de IAM** y **políticas de claves KMS** para restringir quién puede administrar o usar la clave.
- Ejemplo de política restrictiva:

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Effect": "Allow",  
6       "Principal": {"AWS": "arn:aws:iam::123456789012:role/AdminRole"},  
7       "Action": "kms:*",  
8       "Resource": "*"   
9     },  
10    {  
11      "Effect": "Deny",  
12      "Principal": "*",  
13      "Action": "kms:DeleteKey",  
14      "Resource": "*"   
15    }  
16  ]  
17 }
```

3. Monitoreo :

- Usar **AWS CloudTrail** para auditar el uso de la clave (ej: intentos de acceso no autorizados).

Caso práctico

Escenario : Una empresa de salud necesita cifrar datos de pacientes almacenados en un directorio AWS para cumplir **HIPAA** .

- **Solución** :

1. Cree una clave KMS con rotación automática.
2. Asignar la clave al directorio durante su creación.
3. Monitorear el acceso con CloudTrail y generar informes de auditoría.

Resultado : Cumplimiento normativo y protección contra exposición de datos sensibles.

Errores comunes

- **Clave KMS no autorizada** : Si el servicio **ds.amazonaws.com** no tiene permiso para usar la clave, el directorio fallará al crearse.
- **Eliminación accidental de claves** : Bloquear la acción en políticas de IAM. **kms:DeleteKey**

El uso de **AWS KMS** con **AWS Directory Service** añade una capa crítica de seguridad para datos sensibles. Al seguir estas prácticas, garantizas que tu directorio cumpla con los estándares de cifrado y control de acceso, minimizando riesgos de brechas.

Integración con Active Directory y LDAP

¿Por qué integrar Active Directory (AD) con AWS?

Muchas organizaciones utilizan **Active Directory (AD)** local para gestionar usuarios y recursos. Al migrar a la nube, es crítico integrar este directorio con AWS para:

- **Unificar identidades** : Evitar credenciales separadas para entornos on-premises y cloud.
- **Simplificar la gestión** : Aplicar políticas de seguridad y acceso desde un único sistema.
- **Cumplir normativas** : Garantizar autenticación centralizada y auditorías.

Herramientas de Integración en AWS

AWS ofrece dos servicios principales para integrar AD con la nube:

1. **Servicio de directorio de AWS para Microsoft Active Directory (AD administrado) :**
 - Directorio gestionado por AWS, compatible con AD.
 - Ideal para autenticar usuarios en recursos cloud (ej: EC2, RDS).
2. **Conector AD :**
 - Proxy seguro que conecta AD on-premises con AWS sin replicar datos.
 - Útil para acceder a recursos en la nube mediante credenciales locales.

Pasos para integrar AD con AWS

Escenario : Una empresa quiere que sus usuarios locales accedan a instancias EC2 en AWS usando sus credenciales de AD.

Proceso :

1. **Configurar conectividad segura :**
 - Usar **AWS Direct Connect** o **VPN** para enlazar redes locales y AWS.
 - Asegurar resolución DNS entre ambos entornos.
2. **Crear un conector AD en AWS :**
 - Vaya a **AWS Directory Service > Directorios > Crear directorio** .
 - Seleccione **AD Connector** y especifique:
 - **Dominio** : Nombre del AD local (ej:) . **corp.example.com**
 - **Servidores AD** : IP de los controladores de dominio locales.
 - **Contraseña** : Credenciales de un usuario AD con permisos de administrador.
3. **Asociar EC2 a un conector AD :**
 - En la instancia EC2, configure **Dominio** para unirse al directorio a través de AD Connector.
 - Usar **objetos de política de grupo (GPO)** para aplicar políticas de seguridad.

Ejemplo de CLI :

```
1 # Unir una instancia EC2 a AD Connector
2 aws ec2 associate-iam-instance-profile \
3   --instance-id i-1234567890abcdef0 \
4   --iam-instance-profile Name=AD-Connector-Role
```

Integración con LDAP

LDAP (Protocolo ligero de acceso a directorios) es un protocolo para acceder a directorios. AWS Directory Service admite LDAP mediante:

1. **Microsoft AD administrado por AWS :**
 - Compatible con LDAP seguro (LDAPS) sobre TLS.
 - Usar **AWS Certificate Manager (ACM)** para certificados SSL.
2. **Anuncio simple :**
 - Soporte LDAP básico para aplicaciones que no requieren AD completo.

Ejemplo de autenticación LDAP en una aplicación :

```
1 import ldap
2
3 # Conectar al directorio AWS via LDAP
4 conn = ldap.initialize('ldaps://corp.example.com:636')
5 conn.simple_bind_s('user@corp.example.com', 'password')
```

Seguridad en la Integración

- **Cifrado :** Usar **LDAPS** o **Kerberos** para comunicaciones.
- **Firewalls :** Restringir el acceso a puertos críticos (389/LDAP, 636/LDAPS, 88/Kerberos).
- **MFA :** Integrar con **AWS MFA** o **RADIUS** para capas adicionales.

Caso Práctico: Acceso a una Aplicación Web con AD

Escenario : Una aplicación web en EC2 debe autenticar a los usuarios del AD local.

- **Solución :**
 1. Usar **AD Connector** para conectar el AD con AWS.
 2. Configurar la aplicación para usar **LDAP** o **Kerberos**.
 3. Habilitar **MFA** para acceder a la aplicación.

Resultado : Los usuarios acceden con sus credenciales de AD, y la aplicación valida permisos en tiempo real.

Errores comunes

- **DNS no resuelve** : Sin resolución entre AD y AWS, la integración falla.
- **Permisos insuficientes** : El usuario AD debe tener derechos de unión a dominio.
- **Firewalls bloqueando puertos** : Asegurar que los puertos LDAP/Kerberos estén abiertos.

La integración de Active Directory y LDAP con AWS permite una gestión unificada de identidades, mejorando la seguridad y reduciendo la carga administrativa. Herramientas como **AD Connector** y **Managed AD** simplifican este proceso, pero requieren una configuración precisa de red y permisos.

Seguridad en Autenticación y Autorización

Autenticación Segura en AWS

La **autenticación** verifica la identidad de un usuario o servicio, mientras que la **autorización** define qué recursos pueden acceder. En AWS, estos procesos se fortalecen con:

Protocolos de autenticación

1. **LDAP/Kerberos** :
 - **AWS Managed Microsoft AD** utiliza **Kerberos** para autenticación segura de usuarios y servicios.
 - **LDAPS** (LDAP sobre TLS) garantiza comunicaciones cifradas para aplicaciones que usan LDAP.
2. **Autenticación Multifactorial (MFA)** :
 - **Requerido para administradores** : Agregue una capa adicional de seguridad mediante:
 - **Aplicaciones MFA** (ej: Google Authenticator).
 - **Tokens físicos** (ej: YubiKey).
 - **Integración con RADIUS** : Para usar servidores MFA locales.

Ejemplo de habilitación de MFA en IAM :

```
1 # Asociar un MFA virtual a un usuario IAM
2 aws iam enable-mfa-device \
3   --user-name AdminUser \
4   --serial-number arn:aws:iam::123456789012:mfa/AdminUserMFA \
5   --authentication-code1 123456 \
6   --authentication-code2 654321
```

Autorización: Políticas y Roles

La autorización en AWS se basa en:

1. Roles y políticas del IAM :

- **Roles** : Asignar permisos a usuarios, aplicaciones o servicios sin compartir credenciales.
- **Políticas JSON** : Definir permisos (ej: acceso a S3, EC2).

Ejemplo de política de acceso mínimo :

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Effect": "Allow",  
6       "Action": "s3:ListBucket",  
7       "Resource": "arn:aws:s3:::example-bucket"  
8     }  
9   ]  
10 }
```

1. Comparación entre RBAC y ABAC :

- **RBAC (Control de acceso basado en roles)** : Permisos basados en roles (ej: "Admin", "Desarrollador").
- **ABAC (Attribute-Based Access Control)** : Permisos dinámicos basados en atributos (ej: departamento, ubicación).

Configuración de ABAC (Control de acceso basado en atributos) en AWS

Conceptos básicos de ABAC

ABAC es un modelo de control de acceso que otorga permisos basados en **atributos** (propiedades o metadatos) de:

- **Usuarios** (ej: departamento, nivel de seguridad).
- **Recursos** (ej: etiquetas de un bucket S3).
- **Entorno** (ej: hora del día, IP de origen).
- **Acciones** (ej:). **s3:GetObject**

Componentes Clave de ABAC en AWS

1. Atributos :

- **Usuarios/Roles** : Etiquetas (tags) en IAM (ej:). **Department=Finanzas**
- **Recursos** : Etiquetas en recursos AWS (ej:). **Project=Marketing**
- **Entorno** : Variables como región, dirección IP o tiempo.

2. Políticas IAM con Condiciones :

- Usar el elemento en políticas JSON para definir reglas basadas en atributos. **Condition**

Pasos para configurar ABAC

Escenario : Permitir a usuarios del departamento de **Desarrollo** acceder solo a recursos etiquetados con . **Project=AppX**

Etiquetado Consistente

1. Etiquetar usuarios/roles :

- Asignar etiquetas como usuarios de IAM. **Department=Development**

```
1 # Ejemplo CLI: Etiquetar un usuario IAM
2 aws iam tag-user --user-name DevUser --tags Key=Department,Value=Development
```

2. Etiquetar recursos :

- Aplicar etiquetas a recursos como buckets S3, instancias EC2, etc.

```
1 # Ejemplo CLI: Etiquetar un bucket S3
2 aws s3api put-bucket-tagging --bucket appx-bucket --tagging 'TagSet=[{Key=Project,Value=AppX}]'
```


Crear Políticas ABAC con Condiciones

Política de ejemplo :

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Effect": "Allow",  
6       "Action": "s3:*",  
7       "Resource": "arn:aws:s3:::appx-bucket",  
8       "Condition": {  
9         "StringEquals": {  
10          "aws:PrincipalTag/Department": "Development",  
11          "s3:ResourceTag/Project": "AppX"  
12        }  
13      }  
14    ]  
15  }  
16 }
```

Explicación :

- **aws:PrincipalTag/Department** : Verifica que el usuario tenga la etiqueta. **Department=Development**
- **s3:ResourceTag/Project** : Verifique que el recurso tenga la etiqueta **.Project=AppX**

Pruebas con IAM Policy Simulator

1. Acceso similar :

- Usar el **IAM Policy Simulator** para validar si un usuario puede realizar una acción.
- Ejemplo: Verificar si puede acceder a **.DevUserappx-bucket**

2. Resultado esperado :

- **Éxito** : Si coinciden las etiquetas del usuario y el recurso.
- **Fallo** : Si falta alguna etiqueta o no coincide.

Casos de uso práctico

1. Acceso por Proyecto :

- Usuarios de un proyecto solo acceden a recursos etiquetados con su proyecto.

2. Control por horario :

- Restringir acceso a recursos críticos fuera del horario laboral.

```
1 v "Condition": {  
2   "Bool": {"aws:MultiFactorAuthPresent": "true"},  
3   "DateGreaterThan": {"aws:CurrentTime": "2024-01-01T09:00:00Z"},  
4   "DateLessThan": {"aws:CurrentTime": "2024-01-01T18:00:00Z"}  
5 }
```

3. Acceso por Ubicación :

- Permitir acceso solo desde IP corporativas

```
1 v "Condition": {  
2   "IpAddress": {"aws:SourceIp": "203.0.113.0/24"}  
3 }
```

Errores Comunes y Soluciones

1. Etiquetas inconsistentes :

- **Error** : Recursos o usuarios sin etiquetas.
- **Solución** : Usar **AWS Config** para auditar etiquetas.

2. Condiciones demasiado amplias :

- **Error** : Usar sin especificar atributos. **"aws:PrincipalTag/*"**
- **Solución** : Definir condiciones específicas.

3. Ignorar el contexto de seguridad :

- **Error** : No combinar ABAC con MFA o cifrado.
- **Solución** : Integrar múltiples capas de seguridad.

Buenas Prácticas

1. **Mantener etiquetas estandarizadas** : Usar un esquema de etiquetado claro (ej: , ,).
DepartmentEnvironmentProject
2. **Combinar ABAC con RBAC** : Usar roles para grupos grandes y ABAC para permisos granulares.
3. **Monitorear con CloudTrail** : Auditar eventos de acceso para detectar desviaciones.

ABAC en AWS permite un control de acceso dinámico y flexible, ideal para entornos complejos. Su requiere implementación:

- **Etiquetado consistente** de usuarios y recursos.
- **Políticas IAM con condiciones precisas** .
- **Monitoreo continuo** para garantizar eficacia.

Monitoreo y Auditoría

Herramientas clave :

1. **AWS CloudTrail** :
 - Registra eventos de API (ej: inicio de sesión, cambios en IAM).
 - **Ejemplo de consulta en CloudTrail** :

```
1 aws cloudtrail lookup-events --lookup-attributes AttributeKey=Username,AttributeValue=AdminUser
```

2. Amazon CloudWatch :

- Crea alarmas para detectar actividades sospechosas (ej: múltiples intentos fallidos de iniciar sesión).
- **Ejemplo de alarma para intentos fallidos :**

```
1 {  
2   "AlarmName": "High-Frequency-Failed-Logins",  
3   "MetricName": "FailedLogins",  
4   "Threshold": 5,  
5   "ComparisonOperator": "GreaterThanThreshold"  
6 }
```

Seguridad en Servicios de Directorio

1. Servicio de directorio de AWS :

- **Cifrado** : Usar **AWS KMS** para datos en reposo y **TLS** para datos en tránsito.
- **Grupos de seguridad** : Restringir el acceso al directorio solo a IPs o servicios autorizados.

2. Integración con AD :

- **Políticas de Grupo (GPO)** : Aplicar contraseñas complejas y bloqueo de cuentas tras intentos fallidos.

Caso Práctico: Auditoría de Accesos

Escenario : Una empresa detecta accesos no autorizados a un bucket S3.

- **Solución** :
 1. Usar **CloudTrail** para identificar eventos sospechosos.
 2. Revisar políticas IAM asociadas al usuario/rol comprometido.
 3. Ajustar políticas aplicando **privilegio mínimo** y habilitar MFA.

Resultado : Reducción del 90% en incidentes de acceso no autorizado.

Errores comunes

- **Políticas IAM demasiado permisivas** : Ej: Usar en lugar de acciones específicas. "Action": "*"
- **Falta de MFA para roles críticos** : Administradores sin MFA son un vector de ataque común.
- **Ignorar registros de CloudTrail** : No analizar eventos de seguridad periódicamente.

Buenas prácticas

1. **Aplique MFA a todos los usuarios administradores** .
2. **Usar CloudWatch Alarms** para alertar sobre actividades anómalas.
3. **Rotar credenciales regularmente** (ej: claves de acceso IAM).

La seguridad en autenticación y autorización requiere una combinación de protocolos robustos (Kerberos, MFA), políticas precisas (RBAC/ABAC) y monitoreo continuo (CloudTrail/CloudWatch). Estas capas defienden contra accesos no autorizados y cumplen normativas como **ISO 27001** o **SOC 2** .

Análisis de registros con AWS CloudTrail para seguridad IAM

AWS CloudTrail registra eventos de API en tu cuenta AWS, como:

- Inicios de sesión en la consola.
- Cambios en IAM (creación de roles, políticas).
- Accesos a recursos (S3, EC2, etc.).

Estos registros son esenciales para:

- **Auditorías de seguridad** .
- **Detección de actividades sospechosas** (ej: acceso no autorizado).
- **Cumplimiento normativo** (ej: ISO 27001, GDPR).

Configuración Básica de CloudTrail

1. Habilitar CloudTrail :

- Ir a **CloudTrail > Senderos > Crear sendero** .
- Especificar:
 - **S3 Bucket** : Almacenamiento de troncos (ej:). **cloudtrail-logs-example**
 - **Región** : Aplicar a todas las regiones para monitoreo global.
 - **Cifrado** : Usar **AWS KMS** para proteger registros.

2. Integrar con CloudWatch Logs :

- Enviar eventos a **CloudWatch** para análisis en tiempo real.

Ejemplo CLI para crear un sendero :

```
1 aws cloudtrail create-trail \  
2   --name MyTrail \  
3   --s3-bucket-name cloudtrail-logs-example \  
4   --enable-log-file-validation \  
5   --kms-key-id arn:aws:kms:us-east-1:123456789012:key/abcd1234-a123-4567-8abc-def123456789
```

Estructura de un evento CloudTrail

Cada evento es un archivo JSON con campos clave:

- **eventTime** : Fecha y hora del evento.
- **userIdentity** : Quién lo ejecutó (usuario IAM, rol, servicio).
- **eventName** : Acción realizada (ej: ,). **AssumeRoleConsoleLogin**
- **sourceIPAddress** : IP desde donde se originó el evento.
- **ResponseElements** : Resultado de la acción (éxito/error).

Ejemplo de evento de inicio de sesión fallido :

```
1 {
2   "eventVersion": "1.08",
3   "userIdentity": {
4     "type": "IAMUser",
5     "userName": "UnauthorizedUser"
6   },
7   "eventTime": "2024-03-20T14:30:00Z",
8   "eventName": "ConsoleLogin",
9   "sourceIPAddress": "192.0.2.100",
10  "responseElements": {
11    "ConsoleLogin": "Failure"
12  }
13 }
```

Técnicas de Análisis de Registros

Detección de Accesos No Autorizados

Pasos :

1. Filtrar eventos de inicio de sesión fallidos :

```
1 aws cloudtrail lookup-events \
2   --lookup-attributes AttributeKey=EventName,AttributeValue=ConsoleLogin \
3   --query 'Events[?Contains(errorMessage, `Failed`) == `true`]'
```

2. Identificar IPs sospechosas :

1. Buscar IP no asociadas a rangos corporativos.

Monitoreo de Cambios en IAM

Eventos críticos :

- **CreateUser**, **CreateRoleAttachRolePolicy**
- **Ejemplo de consulta :**

```
1 aws cloudtrail lookup-events \
2   --lookup-attributes AttributeKey=EventName,AttributeValue=CreateRole \
3   --start-time 2024-03-01T00:00:00Z
```

Uso de AWS Athena para Análisis Avanzado

Crea una tabla en Athena para consultar registros :

```
1 v CREATE EXTERNAL TABLE cloudtrail_logs (  
2   eventVersion STRING,  
3   userIdentity STRUCT<type:STRING, userName:STRING>,  
4   eventTime TIMESTAMP,  
5   eventName STRING,  
6   sourceIPAddress STRING  
7 )  
8 ROW FORMAT SERDE 'org.openx.data.jsonserde.JsonSerDe'  
9 LOCATION 's3://cloudtrail-logs-example/AWSLogs/123456789012/';
```

Consulta para detectar roles comprometidos :

```
1 v SELECT eventTime, userIdentity.userName, eventName  
2 FROM cloudtrail_logs  
3 WHERE eventName = 'AssumeRole'  
4 AND sourceIPAddress NOT LIKE '203.0.113.%';
```

Casos de uso práctico

Investigación de un incidente de seguridad

Escenario : Un bucket S3 con datos sensibles fue accedido desde una IP desconocida.

- **Análisis con CloudTrail :**
 1. Buscar eventos en el bucket. **GetObject**
 2. Identificar la IP y el usuario/rol involucrado.
 3. Verificar si hubo creación de roles o políticas sospechosas antes del incidente.

Cumplimiento de Políticas de Seguridad

Ejemplo : Asegurar que solo los usuarios del departamento de seguridad pueden eliminar registros.

- **Política IAM :**

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Effect": "Deny",  
6       "Action": "s3:DeleteObject",  
7       "Resource": "arn:aws:s3:::cloudtrail-logs-example/*",  
8       "Condition": {  
9         "StringNotEquals": {  
10          "aws:PrincipalTag/Department": "Security"  
11        }  
12      }  
13    ]  
14  }  
15 }
```

Integración con Herramientas de Seguridad

1. **AWS GuardDuty** : analiza registros de CloudTrail para detectar comportamientos maliciosos.
2. **Amazon Macie** : Identifica datos sensibles accedidos o compartidos.
3. **SIEM (Splunk, ELK)** : Centraliza y correlaciona registros de múltiples fuentes.

Errores Comunes y Soluciones

1. **Registros sin cifrados :**
 - **Solución** : Habilitar **KMS** en el sendero y restringir el acceso al cubo S3.
2. **Demora en la entrega de troncos :**
 - **Solución** : Usar **notificaciones de eventos S3** para alertar cuando lleguen nuevos registros.
3. **Ignorar eventos de "Solo lectura" :**
 - **Solución** : Monitorear acciones como `o` , que pueden indicar reconocimiento de red. **ListBuckets DescribeRoles**

Buenas Prácticas

1. **Habilitar CloudTrail en todas las regiones .**
2. **Validar integridad de registros** (opción). **LogFileValidationEnabled**
3. **Rotar claves KMS** periódicamente.
4. **Usar CloudWatch Alarms** para alertar sobre eventos críticos.

El análisis de logs con **AWS CloudTrail** es fundamental para mantener un entorno cloud seguro. Combinado con herramientas como **Athena** , **GuardDuty** y políticas IAM precisas, permite detectar y responder rápidamente a amenazas, garantizando cumplimiento y protección de recursos críticos.

Seguridad en Autenticación y Autorización

La **autenticación** verifica quién es el usuario o servicio, mientras que la **autorización** define qué puede hacer. En AWS, estos procesos son críticos para proteger recursos y cumplir normativamente.

Mecanismos de Autenticación Segura

1. **Autenticación Multifactorial (MFA) :**
 - **Requerido para roles críticos** : Los usuarios administradores deben usar MFA.
 - **Integración con AWS :**
 - **Virtual MFA** : Aplicaciones como Google Authenticator.
 - **Hardware MFA** : Dispositivos como YubiKey.
 - **Ejemplo de habilitación de MFA:**

```
1 aws iam enable-mfa-device \  
2   --user-name AdminUser \  
3   --serial-number arn:aws:iam::123456789012:mfa/AdminUserMFA \  
4   --authentication-code1 123456 \  
5   --authentication-code2 654321
```

2. **Protocolos de Autenticación :**
 - **Kerberos** : Usado en **AWS Managed Microsoft AD** para autenticación de usuarios.
 - **LDAPS** : LDAP sobre TLS para aplicaciones que requieren cifrado.

Autorización: Políticas y Roles

1. Roles de IAM vs. Usuarios :

- **Roles** : Asignar permisos temporales a servicios (ej: EC2, Lambda).
- **Usuarios** : Credenciales permanentes para personas o aplicaciones.

2. Políticas de Autorización :

- **RBAC (Control de acceso basado en roles)** :
 - Ejemplo: Rol "Desarrollador" con acceso a S3 y CloudWatch.
- **ABAC (Control de acceso basado en atributos)** :
 - Permisos basados en atributos (ej: etiquetas de recursos).

Ejemplo de política RBAC :

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Effect": "Allow",  
6       "Action": "s3:ListBucket",  
7       "Resource": "arn:aws:s3:::example-bucket"  
8     }  
9   ]  
10 }
```

Monitoreo y Auditoría

1. AWS CloudTrail :

- Registra eventos de API (ej: inicio de sesión, cambios en IAM).
- **Ejemplo de consulta :**

```
1 aws cloudtrail lookup-events --lookup-attributes AttributeKey=Username,AttributeValue=AdminUser
```

2. Amazon CloudWatch :

- Crea alarmas para actividades sospechosas (ej: intentos fallidos de inicio de sesión).
- **Ejemplo de alarma :**

```
1 {  
2   "AlarmName": "High-Frequency-Failed-Logins",  
3   "MetricName": "FailedLogins",  
4   "Threshold": 5  
5 }
```

Seguridad en Servicios de Directorio

1. Servicio de directorio de AWS :

- **Cifrado** : Datos en reposo con **AWS KMS** y en tránsito con **TLS** .
- **Grupos de seguridad** : Restringir el acceso al directorio.

2. Integración con Active Directory :

- **Políticas de Grupo (GPO)** : Forzar contraseñas complejas y bloqueo de cuentas.

Caso Práctico: Auditoría de Accesos

Escenario : Un bucket S3 con datos sensibles fue accedido desde una IP desconocida.

- **Análisis con CloudTrail** :
 1. Identificar eventos en el cubo. **GetObject**
 2. Verifique la IP y el usuario/rol involucrado.
 3. Ajustar políticas IAM para restringir el acceso.

Resultado : Reducción del 90% en incidentes de acceso no autorizado.

Buenas Prácticas

1. **Aplique MFA a todos los usuarios administradores** .
2. **Usar el principio de privilegio mínimo** en políticas IAM.
3. **Monitorear registros de CloudTrail** para detectar actividades anómalas.



La seguridad en autenticación y autorización requiere capas defensivas: MFA, políticas precisas (RBAC/ABAC) y monitoreo continuo. Estas prácticas protegen contra accesos no autorizados y cumplen con normativas como **ISO 27001** o **GDPR**.

Configuración de MFA para roles en AWS

Conceptos básicos

- **MFA (Autenticación Multifactor)** : Capa adicional de seguridad que requiere algo que el usuario **sabe** (contraseña) y algo que el usuario **tiene** (token físico o app).
- **Roles en AWS** : Entidades temporales que otorgan permisos a usuarios, aplicaciones o servicios sin necesidad de compartir credenciales permanentes.

¿Por qué usar MFA con roles?

- **Proteger roles críticos** : Ej: Roles con acceso a datos sensibles o servicios como S3, EC2, o IAM.
- **Cumplir normativas** : Requerido en estándares como **SOC 2**, **ISO 27001**, o **PCI-DSS**

Configuración de MFA para Roles

Existen dos escenarios principales:

1. **MFA al asumir un rol en la consola de AWS**.
2. **MFA al asumir un rol programáticamente (CLI/SDK)**.

Escenario 1: MFA en la Consola de AWS

Pasos :

1. Habilitar MFA en el usuario que asumirá el rol :

- Ir a IAM > Usuarios > Seleccionar usuario > **Seguridad de inicio de sesión** > **Asignar MFA** .

2. Crear una política que exija MFA al asumir el rol :

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Effect": "Allow",  
6       "Action": "sts:AssumeRole",  
7       "Resource": "arn:aws:iam::123456789012:role/ExampleRole",  
8       "Condition": {  
9         "Bool": {  
10          "aws:MultiFactorAuthPresent": "true"  
11        }  
12      }  
13    }  
14  ]  
15 }
```

- Esta política permite asumir el rol **solo si el usuario ha autenticado con MFA** .

2. Asignar la política al usuario o grupo IAM :

- Asegurar que el usuario tenga permisos para asumir el rol **y** que la política exija MFA.

Resultado :

- Al intentar asumir el rol en la consola, el usuario deberá ingresar un código MFA.

Escenario 2: MFA en CLI/SDK (Programático)

Pasos :

1. Obtener credenciales temporales con MFA :

- Usar con un código MFA: **aws sts get-session-token**

```
1 aws sts get-session-token \  
2 --serial-number arn:aws:iam::123456789012:mfa/UserMFA \  
3 --token-code 123456
```

- Esto genera credenciales temporales (, ,). **AccessKeyId SecretAccessKey SessionToken**

```
1 aws sts assume-role \  
2 --role-arn arn:aws:iam::123456789012:role/ExampleRole \  
3 --role-session-name "ExampleSession" \  
4 --serial-number arn:aws:iam::123456789012:mfa/UserMFA \  
5 --token-code 123456
```

Nota :

- El parámetro es el ARN del dispositivo MFA del usuario. **--serial-number**
- El es el código generado por la aplicación MFA. **--token-code**

Políticas de Roles que Exigen MFA

Para forzar MFA al asumir un rol, usa la condición en la política del rol: **aws:MultiFactorAuthPresent**

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Effect": "Allow",  
6       "Principal": {"AWS": "arn:aws:iam::123456789012:user/ExampleUser"},  
7       "Action": "sts:AssumeRole",  
8       "Condition": {  
9         "Bool": {"aws:MultiFactorAuthPresent": "true"}  
10      }  
11    }  
12  ]  
13 }
```

Caso Práctico: MFA para un Rol Cross-Account

Escenario : Un usuario de la cuenta **A** necesita asumir un rol en la cuenta **B** , pero se requiere MFA.

Pasos :

1. En la cuenta B :

- Crear un rol con una política que permita acceso a recursos específicos (ej: S3).
- En la política de confianza (), exige MFA: **Trust Policy**

The screenshot shows a web browser window with a Qwen chat interface. The chat history on the left lists several sessions. The main chat area displays a list of steps for setting up a role in account B:

- 1. En la cuenta B :**
 - Crear un rol con una política que permita acceso a recursos específicos (ej: S3).
 - En la política de confianza (), exige MFA: **Trust Policy**
- 2. En la cuenta A :**
 - Asegúrese de que el usuario tenga MFA habilitado y permisos para asumir el rol.

Below the steps, a JSON policy is shown in a code block:

```
1 {
2   "Version": "2012-10-17",
3   "Statement": [
4     {
5       "Effect": "Allow",
6       "Principal": {"AWS": "arn:aws:iam::ACCOUNT_A:user/ExampleUser"},
7       "Action": "sts:AssumeRole",
8       "Condition": {
9         "Bool": {"aws:MultiFactorAuthPresent": "true"}
10      }
11    }
12  ]
13 }
```

The interface includes a search bar, a list of chats, and a chat input area at the bottom with a 'Pensando (QwQ)' status. The Windows taskbar at the bottom shows the date as 10-03-2025 and time as 16:07.

2. En la cuenta A :

- Asegúrese de que el usuario tenga MFA habilitado y permisos para asumir el rol.

Buenas Prácticas

1. **Exigir MFA para roles críticos :**
 - Roles con acceso a IAM, KMS o datos sensibles.
2. **Usar credenciales temporales :**
 - Evitar compartir roles sin MFA en scripts o aplicaciones.
3. **Monitorear con CloudTrail :**
 - Detectar intentos de asumir roles sin MFA:

```
1 aws cloudtrail lookup-events \  
2 --lookup-attributes AttributeKey=EventName,AttributeValue=AssumeRole \  
3 --query 'Events[?errorCode!=`null`]'
```

Errores comunes

1. **Olvidar habilitar MFA en el usuario :**
 - El usuario debe tener MFA configurado antes de asumir el rol.
2. **Políticas mal configuradas :**
 - Si la condición no está presente, el MFA no se exigirá. **aws:MultiFactorAuthPresent**
3. **Credenciales temporales vencidas :**
 - Usar para renovarlas. **aws sts get-session-token**

Configurar MFA para roles en AWS añade una capa crítica de seguridad, especialmente en entornos multicuenta o con acceso programático. Combinar políticas IAM con condiciones de MFA y monitoreo continuo garantiza que solo los usuarios autenticados accedan a recursos sensibles.

Caso Práctico - Seguridad en AWS Directory Service

Escenario

Una empresa migra sus aplicaciones y recursos a AWS y necesita:

- **Autenticar empleados** usando su **Active Directory (AD) local** existente.
 - **Proteger el acceso** a recursos en la nube (ej: EC2, RDS, aplicaciones empresariales).
 - **Cumplir normativas** como GDPR e ISO 27001.
-

Arquitectura Propuesta

Componentes clave :

1. **Servicio de directorio de AWS :**
 - **AWS Managed Microsoft AD** : Directorio en la nube para autenticar usuarios y recursos.
 - **Conector AD** : Proxy seguro para conectar el AD local con AWS.
2. **Relación de confianza (Trust) :**
 - Permite que el AD on-premises y el AWS Managed AD se autenticuen mutuamente.
3. **MFA y Monitoreo :**
 - **AWS MFA** para administradores.
 - **CloudTrail** y **CloudWatch** para auditar accesos.

Implementación Paso a Paso

Crear un AWS Managed Microsoft AD

1. **Configuración inicial :**
 - **Parámetros :**
 - **Clave KMS** : Cifrado de datos en reposo.
 - **VPC/Subredes** : Asegurar alta disponibilidad en múltiples zonas.

```
1 aws ds create-microsoft-ad \  
2   --name corp.example.com \  
3   --password "SecurePassword123!" \  
4   --edition Enterprise \  
5   --vpc-settings VpcId=vpc-12345678,SubnetIds=subnet-abc123,subnet-def456 \  
6   --kms-key-id arn:aws:kms:us-east-1:123456789012:key/abcd1234-a123-4567-8abc-def123456789
```

2. **Verificación :**
 - Confirmar en la consola de AWS que el directorio está en estado Active

Configurar una relación de confianza con AD On-Premises

1. Crear la relación en AWS :

```
1 aws ds create-trust \  
2   --directory-id d-1234567890 \  
3   --remote-domain-name onprem.corp.example.com \  
4   --trust-type Forest \  
5   --trust-direction Two-Way \  
6   --trust-password "SecureTrustPass123!"
```

2. Configurar AD local :

- Usar **dominios y fideicomisos de Active Directory** para sincronizar con el directorio de AWS.

Habilitar MFA para Administradores

1. Asignar MFA a usuarios críticos :

```
1 aws iam enable-mfa-device \  
2   --user-name AdminUser \  
3   --serial-number arn:aws:iam::123456789012:mfa/AdminUserMFA \  
4   --authentication-code1 123456 \  
5   --authentication-code2 654321
```

2. Política IAM para exigir MFA :

```
1 {  
2   "Version": "2012-10-17",  
3   "Statement": [  
4     {  
5       "Effect": "Deny",  
6       "Action": "*",  
7       "Resource": "*",  
8       "Condition": {  
9         "BoolIfExists": {  
10          "aws:MultiFactorAuthPresent": "false"  
11        }  
12      }  
13    }  
14  ]  
15 }
```

Monitoreo con CloudTrail y CloudWatch

1. Crear una alarma en CloudWatch :

```
1 {  
2   "AlarmName": "UnauthorizedAccessAttempts",  
3   "MetricName": "FailedLogins",  
4   "Namespace": "AWS/CloudTrail",  
5   "Statistic": "Sum",  
6   "Period": 300,  
7   "Threshold": 5,  
8   "ComparisonOperator": "GreaterThanThreshold"  
9 }
```

2. Consultar eventos críticos con CLI :

```
1 aws cloudtrail lookup-events \  
2   --lookup-attributes AttributeKey=EventName,AttributeValue=AssumeRole \  
3   --query 'Events[?errorCode!=`null`]'
```

Resultados

- **Acceso unificado** : Los usuarios acceden a recursos en la nube y on-premises con las mismas credenciales.
- **Reducción de riesgos** :
 - **90% menos intentos de acceso no autorizado** gracias a MFA y políticas de confianza.
 - **Cumplimiento normativo** : Auditorías simplificadas con CloudTrail.

Lecciones aprendidas

1. DNS y conectividad :

- Resolución de nombres entre AD on-premises y AWS es crítica para la relación de confianza.

2. Cifrado y MFA :

- KMS y MFA son esenciales para proteger datos sensibles.

3. Monitoreo proactivo :

- CloudWatch Alarms permite responder rápidamente a las amenazas.

Este caso práctico demuestra cómo integrar un AD on-premises con AWS Directory Service de forma segura, usando relaciones de confianza, MFA y monitoreo continuo. La combinación de estos elementos garantiza una gestión de identidades robusta y escalable en entornos híbridos.

Conclusión

Resumen de los Conceptos Clave

En esta sesión, se abordarán estrategias y herramientas esenciales para proteger servicios de **Identity and Access Management (IAM)** en AWS, con foco en:

1. Servicios de directorio en la nube :

- **AWS Directory Service** : configuración de directorios gestionados (Microsoft AD, Simple AD) e integración con sistemas locales.
- **Relaciones de confianza** : Sincronización segura entre AD on-premises y AWS.

2. Autenticación y autorización :

- **MFA obligatorio** para usuarios administradores.
- **ABAC y RBAC** : Políticas basadas en roles y atributos para acceso granular.

3. Monitoreo y auditoría :

- Uso de **AWS CloudTrail** y **CloudWatch** para detectar actividades sospechosas.

Logros del Módulo

- **Protección de recursos críticos :**
 - Cifrado con **AWS KMS** y control de acceso mediante **Security Groups** .
 - Integración segura con **Active Directory** para entornos híbridos.
- **Cumplimiento normativo :**
 - Políticas IAM alineadas con estándares como **GDPR** , **ISO 27001** y **SOC 2** .
- **Reducción de riesgos :**
 - Detección temprana de accesos no autorizados mediante análisis de registros.

Caso Práctico Recapitulado

Se demuestra cómo una empresa migra sus servicios a AWS y protege sus recursos mediante:

1. **AWS Managed Microsoft AD** : Autenticación unificada para usuarios on-premises y cloud.
2. **MFA y políticas de confianza** : Evita accesos fraudulentos.
3. **Monitoreo continuo** : Alarmas en CloudWatch y auditorías con CloudTrail.

Resultado :

- **80% menos incidentes de seguridad** relacionados con acceso a datos.
- **Simplificación de la gestión** de identidades en un entorno híbrido.

Lecciones Clave para los Alumnos

1. **Seguridad desde el diseño :**
 - Aplicar **privilegio mínimo** y **MFA** desde la creación de roles y políticas.
2. **Integración híbrida :**
 - Usar **AD Connector** o **relaciones de confianza** para conectar entornos locales y en la nube.
3. **Automatización y monitoreo :**
 - Usar **AWS Lambda** para respuestas automáticas ante eventos sospechosos.



Recomendaciones finales

- **Documentar políticas** : Mantener un registro claro de roles, políticas y relaciones de confianza.
- **Capacitación continua** : Explorar herramientas como **AWS IAM Identity Center** para gestión avanzada.
- **Pruebas de penetración** : Validar la seguridad de configuración IAM periódicamente.

La seguridad en IAM no es un punto final, sino un proceso continuo que requiere:

- **Actualización constante** ante nuevas amenazas.
- **Combinación de herramientas nativas de AWS** (KMS, CloudTrail) y buenas prácticas (ABAC, MFA).
- **Enfoque proactivo** para proteger identidades, datos y recursos en la nube.

Este módulo proporciona las bases para diseñar arquitecturas en la nube seguras, preparando a los alumnos para desafíos reales en entornos empresariales.